T8. Patrones de creación

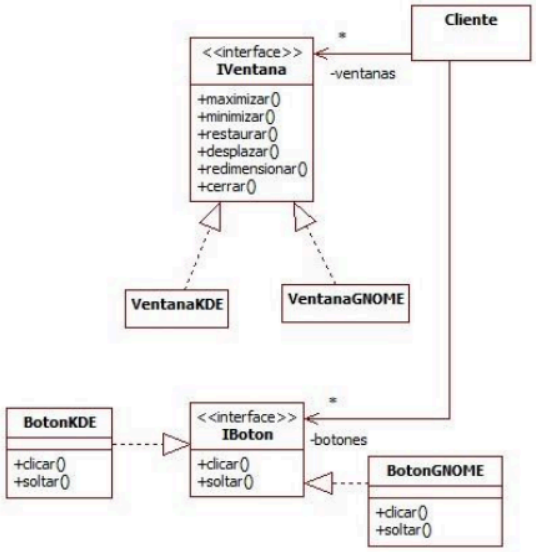
Patrones de creación

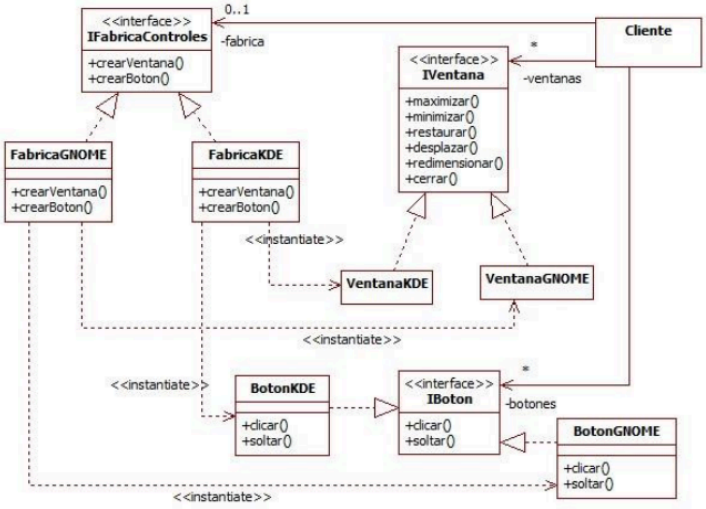
Los **patrones de creación** permiten abstraerse de la creación de objetos, acotando el conocimiento sobre las clases concretas utilizadas. Ocultan cómo se crean y enlazan las instancias de las clases, de manera que el resto del sistema sólo conoce los objetos a través de sus interfaces. Hacen posible programar para **interfaces** y no para implementaciones.

Abstract Factory (Fábrica Abstracta)

La abstract factory aporta una **interfaz** para crear **familias** de objetos ocultando clases concretas. Se usa para:

- Configurar un sistema escogiendo una sola familia de **productos** entre varias.
- Si cada **familia** está diseñada para el uso conjunto de sus productos.
- Para proporcionar **bibliotecas** de productos de los que sólo se revelan sus interfaces.





En el primer dibujo ves dos ideas clave: **interfaces de producto** y **clases concretas** que las implementan. Por un lado está **IVentana**, que define qué puede hacer cualquier ventana (maximizar, minimizar, mover, redimensionar, cerrar). Por otro, **IBoton**, que define las acciones básicas de un botón (clicar y soltar). Estas interfaces son contratos: dicen *qué* se puede hacer, pero no *cómo* se hace. Debajo aparecen las implementaciones concretas: VentanaKDE y VentanaGNOME implementan IVentana, mientras que BotonKDE y BotonGNOME implementan IBoton. Aquí ya se ve la idea de **familias**: KDE por un lado y GNOME por otro. Cada familia mantiene coherencia visual y de comportamiento.

El **Cliente** solo conoce las interfaces (IVentana e IBoton) y mantiene colecciones de ventanas y botones (puede trabajar con *cualquier* ventana o botón sin importarle si es KDE o GNOME).

El segundo diagrama completa la historia introduciendo **IFabricaControles**. Esta es la **fábrica abstracta**. Define dos métodos: crearVentana() y crearBoton(). No dice qué tipo exacto devuelve, solo que serán objetos que cumplen las interfaces (IVentana e IBoton). Luego aparecen dos fábricas concretas: FabricaKDE y FabricaGNOME. Cada una implementa IFabricaControles, pero su comportamiento es distinto:

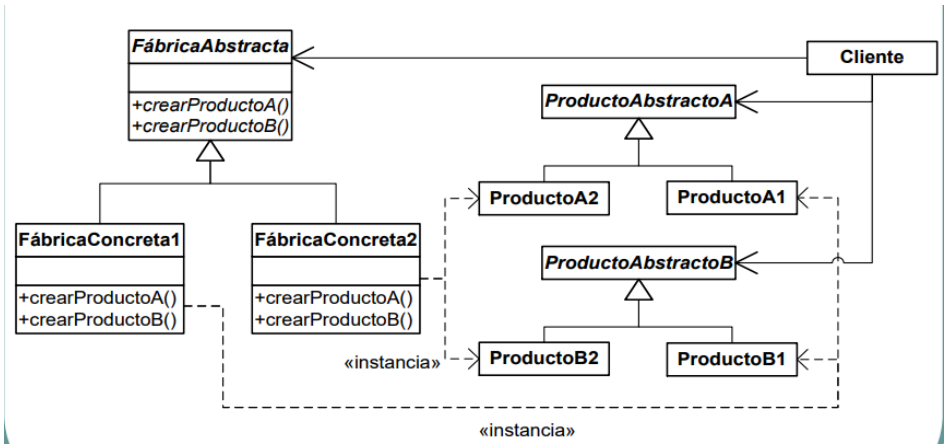
- FabricaKDE crea VentanaKDE y BotonKDE.
- FabricaGNOME crea VentanaGNOME y BotonGNOME.

Las flechas con «instantiate» indican precisamente qué clase concreta crea cada fábrica. Aquí está la clave del patrón: **cada fábrica produce una familia completa y consistente de objetos**. El Cliente ahora tiene una referencia a una IFabricaControles (no a una fábrica concreta).

- <u>Participantes</u>

- **FabricaAbstracta**: declara una interfaz para operaciones que crean produtos abstractos.
- **FabricaConcreta**: implementa operaciones para crear produtos concretos.
- **ProdutoAbstracto**: declara una interfaz para un tipo de producto.
- **ProdutoConcreto**: define un produto a ser creado por la fábrica correspondiente (implementa ProductoAbstracto).
- **Cliente**: usa objetos a través de interfaces.

- <u>Estructura</u>



En la parte superior izquierda se encuentra la **FábricaAbstracta**, que actúa como un contrato general para la creación de objetos. Declara los métodos

crearProductoA() y crearProductoB(), estableciendo que cualquier fábrica que la implemente debe ser capaz de generar esos dos tipos de productos. A partir de esta fábrica abstracta surgen dos implementaciones concretas: **FábricaConcreta1** y **FábricaConcreta2**. Ambas heredan la estructura definida por la fábrica abstracta, pero se diferencian en los objetos específicos que crean. Cada una implementa los métodos de creación y se encarga de instanciar productos concretos.

En el lado derecho del diagrama aparece el **Cliente**, que interactúa con todo el sistema pero de manera desacoplada. Este no depende de clases concretas, sino que trabaja únicamente con la **FábricaAbstracta** y con los productos en su forma abstracta. Es decir, el cliente solicita objetos a la fábrica sin saber qué implementación concreta recibirá, y luego los utiliza a través de sus interfaces abstractas.

En el centro del diagrama se distinguen dos jerarquías de productos. Por un lado está **ProductoAbstractoA**, que define la estructura común para un tipo de producto, y que tiene como implementaciones a **ProductoA1** y **ProductoA2**. Por otro lado está **ProductoAbstractoB**, que cumple el mismo papel para otro tipo de producto, con implementaciones concretas **ProductoB1** y **ProductoB2**.

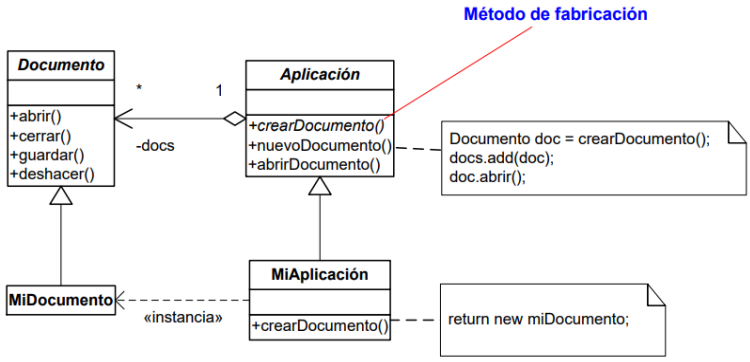
Las relaciones indicadas con **líneas punteadas** y la etiqueta «instancia» muestran qué objetos crea cada fábrica concreta. **FábricaConcreta1** está asociada con la creación de **ProductoA1** y **ProductoB1**, mientras que **FábricaConcreta2** crea **ProductoA2** y **ProductoB2**.

- <u>Ventajas e inconvenientes</u>

El **desacoplamiento** entre **clientes** y **productos**, los clientes manipulan las instancias a través de las interfaces. La facilidad de cambio de familias y la consistencia, las aplicaciones sólo generan productos de la misma familia. Sin embargo, introduce una problemática de añadir nuevos tipos de productos.

Factory Method (Método de fabricación)

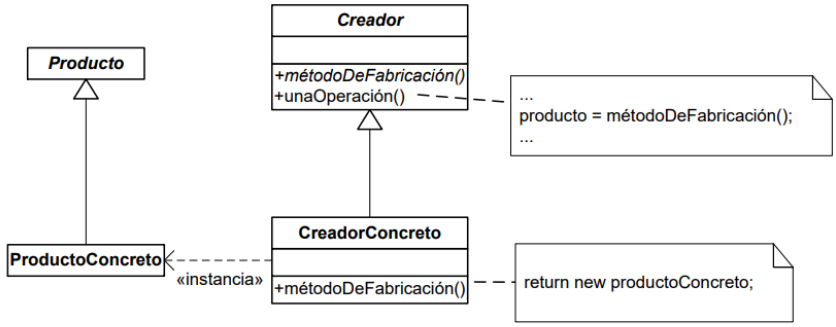
Define una **interfaz** para **crear objetos** cediendo a las **subclases** la decisión sobre qué clase instanciar, es decir, permite que una clase deje en manos de sus subclases la creación de productos. Se usa cuando una clase **no** puede **prever** la clase de objetos que debe crear.



- <u>Participantes</u>

- **Producto**: define la interfaz de los objetos creados por el método de fabricación.
- **ProductoConcreto**: implementa la interfaz Producto.
- **Creador**: declara (y puede implementar) el método de fabricación, el cual devuelve un objeto Producto.
- **CreadorConcreto**: redefine el método de fabricación para devolver una instancia de ProductoConcreto.

- <u>Estructura</u>



En la parte izquierda aparece **Producto**, que es una clase o interfaz abstracta: no se puede instanciar directamente, pero define qué operaciones deben tener los objetos que se creen. Debajo está **ProductoConcreto**, que hereda de Producto y representa una implementación real; es el objeto que finalmente se va a instanciar. En el centro del diagrama está **Creador**, otra clase abstracta que declara el método clave llamado *métodoDeFabricación()*. Este método no crea directamente el objeto, sino que define la firma (qué devuelve) y delega la creación a las subclases. Además, Creador puede tener otros métodos, como *unaOperacion()*, que utilizan el producto sin saber exactamente qué clase concreta es; esto es importante porque desacopla el uso del objeto de su creación. Debajo de Creador aparece **CreadorConcreto**, que hereda de él y es quien realmente implementa *métodoDeFabricación()*, devolviendo una instancia de ProductoConcreto (por ejemplo, "return new productoConcreto").

Las flechas indican herencia: ProductoConcreto extiende Producto, y CreadorConcreto extiende Creador. La relación clave es que Creador no conoce la clase exacta que se va a instanciar; solo sabe que recibirá un objeto de tipo Producto, mientras que CreadorConcreto decide cuál crear.

- <u>Ventajas e inconvenientes</u>

Los clientes sólo tratan con la **interfaz** Producto y pueden tener que **heredar** de Creador simplemente para crear un determinado ProductoConcreto. Dota a subclases de un enganche para proveer **versión extendida** de un objeto y puede conectar jerarquías paralelas.

Comparación entre patrones

Existen dos formas de **parametrizar** un sistema:

- **Heredar** de la **clase** que **crea** los objetos (Factory Method).
- **Definir** un **objeto** responsable de **conocer** la clase de los productos (Abstract Factory)

Factory Method hace más **adaptable** el diseño a cambio de un poco de complejidad, mientras que Abstract Factory da lugar a un diseño aún más **flexible** y **complejo.**